

# claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

## Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde
xer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_ef85c152-821\agent-
a5d3d8c045b07803b.jsonl`
- SHA-256: `9d8d9950e674ab5e355533340ee007ad860d146e81fc460b4649272505e9e747`
- Source modified: `2026-06-16T06:37:48+00:00`
- Imported at: `2026-07-05T16:48:29+00:00`
- project: `wf_ef85c152-821`
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

## Transcript

### 1. user (2026-06-16T06:35:52.032Z)

Triage the audit findings + their skeptical verifications into mustFix (CONFIRMED real, fix now), niceToHave (real but minor/deferable), and nonIssues (refuted / already-handled / by-design). Give an overall verdict on whether the SHIPPED golden-fixtures feature passes due-diligence. Only put CONFIRMED (isMustFix=true) items in mustFix; be concise and concrete (each entry actionable, with the file/function).

AUDITS (JSON):

```
[
  {
    "lens": "DETERMINISM / cross-OS (Golden Regression Fixtures, PRs #186/#189/#191/#192,
worktree gf-doc @ 8d3e185)",
    "summary": "The COMMITTED synthetic CONTROL path is genuinely cross-OS deterministic and the
per-PR verification of it holds: (frame/fps)*fps round-trips exactly for these fixtures so
net_crossings is a stable integer count; JSON writers use sort_keys=True; compare_expected is
dict/set-order-independent and covers every written field; CRLF is fully handled by
csv.reader(newline='')+json.load+the eol=lf .gitattributes pin. The lens's specific
net_crossings/frame*fps and iteration-order suspicions are clean ? I say so honestly. BUT the
lens's central suspicion is correct and uncovered a real defect cluster on the OTHER path. The
de-attribution path (freeze_real_fixture / time-origin reset / M2) is exercised ONLY by
tests/test_golden_real_fixtures.py, which the golden-crossos CI matrix does NOT run (it runs
--check + test_golden_fixtures.py only); that test file runs solely on the ubuntu full-suite
lane, so the reset path has ZERO cross-OS coverage. And inside that under-covered path the
documented 'provably metric-invariant / bit-identical scores' guarantee for the time-origin
reset is FALSE for any non-whole-second t0: I empirically broke it on 26 of 40 sub-second frame
offsets, with a worst score delta of 1.0000000000001e-06 ? landing exactly past the 1e-6
compare_expected tolerance cliff. The committed invariance test only passes because it hard-
codes a whole-second (9000-frame/300.0s) offset that dodges the rounding cliff. Separately, a
real fixture whose GT label starts before its first trajectory frame produces a negative reset
label-start that the STRICT re-parse rejects, crashing freeze_real_fixture AFTER it has already
written trajectory/labels/provenance to disk (orphaned partial fixture). Finally, the 'pure-
stdlib / no numpy on the CONTROL path' framing over-claims for IMPORT: importing golden_fixtures
fails on a clean clone without numpy (experiment->classifier imports numpy at module top) ? only
numpy EXECUTION is avoided.",
    "findings": [
      {
        "title": "Time-origin reset is NOT metric-invariant for non-whole-second t0 ?
contradicts the 'provably metric-invariant / bit-identical scores' claim, and the worst delta
lands past the 1e-6 cliff",
        "severity": "high",
        "evidence": "docs/GOLDEN_REGRESSION_FIXTURES.md:94 ('Time-origin reset ... Provably
metric-invariant ... bit-identical scores'); backend/eval/golden_fixtures.py:619-623 + :583-586
assert the same. But the reset subtracts t0_sec in float and writes labels/trajectory at 6dp
(_fmt, golden_fixtures.py:195-207,274-279), then the STRICT gt_loader re-parses the rounded text
? so when t0_sec is non-terminating, the reset vs no-reset scores diverge. I swept frame offsets
```

0..39 (fps=30) through freeze\_real\_fixture(time\_origin\_reset=True vs False): score equality BROKE on 26/40 offsets; e.g. offset=7 gives mean\_start\_error 0.033333 (reset) vs 0.033334 (no reset). Worst measured  $|\Delta| = 1.000000000001e-06$ , i.e. STRICTLY greater than  $\_FLOAT\_TOL = 1e-6$  (golden\_fixtures.py:91) used by compare\_expected.approx (golden\_fixtures.py:786-789) ? so this can flip a parse-compare digit/result. The committed test only passes because it hard-codes offset\_frames=9000 ?  $9000/30=300.0s$  exactly (tests/test\_golden\_real\_fixtures.py:175), a whole-second offset that never hits the cliff.",

"recommendation": "Either (a) weaken the doc/code claim to 'metric-invariant up to the 6dp serialization rounding' and prove the bound is  $< \_FLOAT\_TOL$ , or (b) make it actually invariant: compute  $t_0$  so the shift is an exact 6dp / exact-frame quantity (e.g. reset by an integer number of frames only and derive  $t_{0\_sec}$  consistently, or round  $t_{0\_sec}$  to 6dp before subtracting from both streams). Add a parametrized test over fractional-second offsets (e.g. 7,11,13 frames) asserting score equality AND  $|\Delta| \leq \_FLOAT\_TOL$ , replacing/augmenting the single 9000-frame case."

},

{

"title": "De-attribution / time-origin-reset path has ZERO cross-OS coverage ? golden-crossos CI omits tests/test\_golden\_real\_fixtures.py (the lens's KEY suspicion, confirmed)",

"severity": "medium",

"evidence": ".github/workflows/ci.yml:119-122 ? the golden-crossos matrix (ubuntu/windows/macOS) runs only `python -m backend.eval.golden\_fixtures --check` and `pytest tests/test\_golden\_fixtures.py`. tests/test\_golden\_real\_fixtures.py (freeze\_real\_fixture, M2 refusal gate, time-origin-reset invariance, strict-loader) is run ONLY by the ubuntu-only full-suite lane via run\_all\_tests.py pytest discovery (ci.yml:67-97). So every de-attribution/reset assertion ? including the metric-invariance claim above ? is verified on ubuntu alone. The doc markets this exact path as the safe-to-publish real-fixture pipeline (P1/P3) and §9 DoD requires 'win+ubuntu+mac' green, but the float-sensitive reset arithmetic that path performs is never run on win/mac.",

"recommendation": "Add tests/test\_golden\_real\_fixtures.py to the golden-crossos matrix step (it is fully synthetic/tmp\_path, no GPU/network ? see its module docstring), so the reset round-trip is exercised on all three OSes. This is also what surfaces finding #1 cross-OS before a real P3 fixture is committed."

},

{

"title": "freeze\_real\_fixture crashes after partial on-disk writes when a GT label precedes the first trajectory frame (negative reset start rejected by STRICT re-parse)",

"severity": "low",

"evidence": "backend/eval/golden\_fixtures.py:624-630 sets  $t_{0\_frame} = \min(\text{frame})$  and  $t_{0\_sec} = t_{0\_frame}/\text{fps}$ , then  $\_reset\_labels$  subtracts  $t_{0\_sec}$  from every label (golden\_fixtures.py:552-554). If a label starts before the first trajectory frame, the reset start goes negative; the writes happen at golden\_fixtures.py:635-636 (trajectory.csv, labels.csv) and :671 (provenance.json) BEFORE replay\_fixture(fdir) at :673, whose strict load\_gt\_intervals raises ValueError('negative start') (gt\_loader.py:78-79). I reproduced it: a synthetic source with trajectory shifted to frame 100 + a label at  $t=0$  fails with 'labels.csv:2: negative start (-3.333333)', leaving trajectory.csv/labels.csv/provenance.json orphaned in the output dir. No cleanup-on-failure and no pre-write validation that all reset labels stay  $\geq 0$ .",

"recommendation": "Validate before writing: after computing reset\_gts, raise RefusalError/ValueError if any reset start  $< 0$  (label timeline predates the trajectory ? a labeling/time-base mismatch), and/or write into a temp dir that is only promoted on success so a failed freeze leaves no partial fixture directory."

},

{

"title": "'Pure-stdlib / no numpy on the CONTROL path' over-claims for IMPORT ? golden\_fixtures cannot be imported on a clean clone without numpy",

"severity": "low",

"evidence": "golden\_fixtures.py:34-35 docstring ('pulls in no numpy/cv2 on the CONTROL path') and eval\_baselines/fixtures/README.md ('CONTROL / pure-stdlib only ? no numpy'). But golden\_fixtures.py:51 does `from backend.eval.experiment import CONTROL, VideoCandidates, predict`, experiment.py:49 does `from backend.eval.classifier import LogisticRegression`, and classifier.py:18 does `import numpy as np` at module top. I simulated a clean clone by blocking numpy import: `from backend.eval import golden\_fixtures` raises ModuleNotFoundError: No module named 'numpy'. CI stays green only because golden-crossos explicitly `pip install ... numpy` (ci.yml:118) and even comments 'CONTROL replay is numpy-only'. So the accurate statement is the already-verified one (numpy is not EXECUTED), not 'no numpy' / 'pure-stdlib'.",

"recommendation": "Correct the docstring/README to 'avoids numpy EXECUTION on the replay

```

path (numpy is still imported transitively via experiment->classifier and must be installed)',
matching ci.yml's own numpy install, OR break the import chain (import LogisticRegression lazily
inside experiment.predict's learned branch) if a truly numpy-free import is desired. Low risk
today since CI installs numpy, but the doc/README claim is false as written."
}
],
"coverageGaps": [
  "Cross-OS execution of the reset path on real Windows/macOS runners was not possible from
this Linux-style dev box; finding #1's 1e-6 cliff is demonstrated as a reset-vs-no-reset
divergence ON ONE MACHINE (a genuine over-claim + a latent cross-OS flake source for future real
fixtures), not as an observed Windows-vs-macOS byte difference on the COMMITTED synthetic
fixtures (those are integer-exact and safe).",
  "No real Fork-A fixture exists yet (P3 gated), so finding #1/#3 are demonstrated on
synthetic sources fed through freeze_real_fixture; they describe what happens WHEN a real
fractional-start fixture is frozen, which is the documented intended use.",
  "Did not re-run the full 759-test suite or coverage here (already-verified: 759 pass/7
skip, 80.49%, ruff+mypy clean); findings are static + targeted-repro, not a suite re-run.",
  "The M2 refusal gate, source-path/MD5/FORBIDDEN_KEYS leak scan (_scan_forbidden),
surrogate-slug randomness, and manifest non-clobber were reviewed and reproduce as designed ? no
defect found there (the schema-literal exclusion at golden_fixtures.py:495-535 correctly avoids
the 'traj' ? 'trajectory.csv' false positive).",
],
"verdict": "NOT CLEAN ? one HIGH over-claim/determinism defect plus a confirming CI coverage
gap. The committed synthetic CONTROL path the per-PR work verified is genuinely cross-OS-safe
(lens's net_crossings/frame*fps/iteration-order suspicions are clean). But the lens's central
suspicion holds: the de-attribution/time-origin-reset path is cross-OS-UNguarded (golden-crossos
omits test_golden_real_fixtures.py; it runs ubuntu-only), and that path's headline 'provably
metric-invariant / bit-identical scores' guarantee is FALSE for non-whole-second t0 ? broken on
26/40 sub-second offsets with a worst delta (1.0000000000001e-06) sitting exactly past the 1e-6
compare cliff, masked by the test's whole-second offset. Plus a partial-write crash on pre-
trajectory labels and a 'no numpy/pure-stdlib' import over-claim. None of these break the
currently-committed synthetic fixtures, but they undermine the doc's safety claims for the real-
fixture (P3) path and the §9 'win+ubuntu+mac' DoD. Fix finding #1 (claim/arithmetic) and #2 (add
real-fixtures test to the cross-OS matrix) before any real Tier-0 fixture is frozen."
},
{
  "lens": "PRIVACY / de-attribution ? completeness of `_scan_forbidden`, surrogate non-
reversibility, refusal-gate bypass, raw-trajectory fingerprinting, and honesty of the doc's
bounded claims, for the shipped `freeze_real_fixture` de-attribution path.",
  "summary": "The audit found the refusal gate (M2) and surrogate slug are genuinely sound ?
the `is not True` checks resist truthy-but-not-True bypass, the `fx<16hex>` slug is os.urandom-
derived with no algebraic link to the source, and no surrogate?source map is written on-git. BUT
the de-attribution scan has two real holes the per-PR verification missed. (1) `_scan_forbidden`
is structurally INCOMPLETE: it catches only biometric key-substrings, a 32-hex MD5, and source-
PATH tokens ? it has ZERO defense against free-text identifiers in the operator-supplied
`source_note` and `label` fields, which are written verbatim into the committed
`provenance.json` (proven: `\"Lin Dan vs Lee, India Open 2024, Court 1\" + \"mens singles final\"`
both committed, no refusal). (2) The hardest concern the lens names ? the raw per-frame shuttle
`(x,y)` trajectory (de Montjoye fingerprint) ? is NOT minimized at all: `freeze_real_fixture`
commits the COMPLETE per-frame `trajectory.csv` with X/Y passed through untouched, directly
contradicting the doc's §4c `\"Raw-trajectory exclusion ? never commit the per-frame CSV\"` and
TM5's `\"only ~5 invariant scalars/window.\"` Two secondary gaps: the post-write scan leaves
leaked files on disk on refusal (no cleanup), and the scan never inspects the written
`trajectory.csv`/`labels.csv` themselves.",
  "findings": [
    {
      "title": "`_scan_forbidden` does NOT catch free-text identifiers in
`source_note`/`label` ? they are committed verbatim into provenance.json",
      "severity": "high",
      "evidence": "golden_fixtures.py:501-535 `_scan_forbidden` scans `text` only for (a)
FORBIDDEN_KEYS biometric substrings, (b) a 32-hex MD5, (c) source-PATH tokens
(sp/p.name/p.stem). `source_note` (line 663) and `label` (line 643, default `\"cleared_real\"`)
are written verbatim into provenance.json and THEN scanned (lines 680-681), but none of the
three categories matches a free-text identifier. Proven empirically: freeze_real_fixture(...,
label='mens singles final', source_note='Lin Dan vs Lee, India Open 2024, Court 1') FROZE with

```

NO refusal and committed both strings into provenance.json. The earlier-passing test test\_golden\_real\_fixtures.py:155 only catches a leak because it passes `source\_note=str(traj)` ? a PATH token the scan does catch ? masking the free-text hole. The lens's question 'a caller-supplied identifying slug, an identifying source\_note' is answered: source\_note is NOT covered.",

"recommendation": "The scan cannot validate free text, so the contract must be enforced differently: (a) hard-restrict `source\_note`/`label` to an allow-list charset/length or a controlled vocabulary, or drop them from the committed provenance entirely (move any human note to the off-git surrogate map); and (b) add a test that a free-text identifying source\_note/label is either rejected or never persisted. At minimum, document loudly in the CLI `--source-note`/`--label` help and in §4c that these fields are committed UNSCANNED and the operator is solely responsible ? the current help says 'NEVER a path/name/id' but nothing enforces it for names/ids."

},

{

"title": "Raw per-frame `(Frame,Visibility,X,Y)` trajectory is committed verbatim for real fixtures ? the de Montjoye spatial fingerprint is fully retained, contradicting doc §4c/TM5",

"severity": "high",

"evidence": "freeze\_real\_fixture writes the FULL per-frame trajectory: golden\_fixtures.py:635 `write\_trajectory\_csv(rows, fdir/'trajectory.csv')` where rows come from `\_reset\_trajectory\_rows` (lines 538-549) which passes X/Y through UNTOUCHED ('Visibility/X/Y are passed through untouched'), shifting only the frame index. The committed clean\_single\_rally/trajectory.csv shows full per-frame X/Y (e.g. `1,1,723.164676,372.202099`). This directly contradicts docs/GOLDEN\_REGRESSION\_FIXTURES.md §4c 'Raw-trajectory exclusion ? never commit the per-frame (Frame,Visibility,X,Y) CSV' and §4b/TM5 'only ~5 invariant scalars/window'. The time-origin reset severs the absolute-TIMELINE correlation but does nothing to the SPATIAL fingerprint the lens explicitly flags (de Montjoye: 4 points ? 95% re-ID). The whole expected.json minimization-to-5-scalars is undercut because the raw CSV sits right next to it in the same tracked directory.",

"recommendation": "Either (a) make the design match the doc: do NOT commit trajectory.csv for cleared\_real fixtures ? derive expected.json on the GPU/owner box and commit only the aggregate snapshot, with replay reading from a non-committed/Tier-1 trajectory; or (b) if the raw CSV must be committed for replay, FIX the doc (§4c, TM5, TL;DR 'publish shuttle-only, drop person/pose') to state plainly that the full per-frame shuttle trajectory IS published, re-do the §3/§5 non-attributability analysis on THAT basis (the de Montjoye fingerprint argument applies to the committed coords, not just to a withheld CSV), and confirm the owner accepted THAT risk ? the 2026-06-16 acceptance was made against a doc that claims the raw CSV is excluded."

},

{

"title": "Post-write privacy scan leaves leaked files on disk on refusal (no cleanup); gate-vs-scan cleanup asymmetry",

"severity": "medium",

"evidence": "golden\_fixtures.py:632-684: the four artifacts (trajectory.csv, labels.csv, expected.json, provenance.json) are written (lines 635-674) BEFORE `\_scan\_forbidden` runs (lines 678-681). When the scan raises RefusalError the function aborts with no try/finally cleanup. Proven empirically: a refused freeze leaves

`['expected.json','labels.csv','provenance.json','trajectory.csv']` in fdir on disk. Contrast the M2 refusal gate (lines 591-605), which fires BEFORE any write ? and whose test (test\_golden\_real\_fixtures.py:84) explicitly asserts 'a refused freeze must not write any fixture'. The post-write scan has no equivalent guarantee and test\_positive\_privacy\_assertion\_catches\_leak\_in\_source\_note (line 155) asserts only that RefusalError is raised, never that the (potentially leak-bearing) files were removed. A leaked provenance.json containing the very PII the scan rejected is left on the owner's disk where a later `git add eval\_baselines/fixtures` could sweep it in.",

"recommendation": "Wrap the write+scan in try/except and on RefusalError remove the partially-written fixture dir (or write to a temp dir and atomically move into place only after the scan passes). Add a regression test asserting the fixture dir does not exist after a scan-triggered refusal."

},

{

"title": "The de-attribution scan never inspects the written `trajectory.csv` / `labels.csv` themselves",

"severity": "low",

"evidence": "golden\_fixtures.py:678-681 calls `\_scan\_forbidden` on expected.json and provenance.json ONLY. trajectory.csv and labels.csv (the files that actually carry the raw coordinate fingerprint and any source-derived row content) are never scanned. In the current flow these CSVs are written from parsed numeric rows so they cannot carry a filename/key, but the positive-assertion guard the doc sells (§4c red-team ? 'positive privacy assertion re-reads the written output') does not actually cover the two largest committed files. This is a coverage gap in the guard's claimed scope, not an active leak today.",

"recommendation": "Extend the positive scan to also read trajectory.csv/labels.csv (cheap), so the 'we re-read the written output and refuse if anything attributable slipped in' claim is true for the whole committed fixture, not just the two JSONs. This also future-proofs against a schema change that ever embeds a non-numeric token in a CSV cell."

},  
{

"title": "fps / frame\_width committed un-quantized as a camera/venue quasi-identifier (documented-but-shipped gap)",

"severity": "low",

"evidence": "golden\_fixtures.py:647-648 writes `\_round(fps)` / `\_round(frame\_width)` (6dp rounding, NOT venue-quantization) into provenance.json, and `\_manifest\_entry\_for` (lines 693-694) copies them into the tracked manifest.json. docs §4c lists 'Quantize fps/frame\_width ? at n=6 the raw values are a camera/venue quasi-identifier' as an anti-attribution measure, but §7 ? honestly admits 'fps/fw retained as scale constants; quantize is a P3 option'. So the quasi-identifier IS committed; this is disclosed in the tracker but the §4c bullet reads as if it were implemented. The lens explicitly asks whether fps/frame\_width-as-quasi-identifier is caught ? it is not (by design, deferred).",

"recommendation": "Reconcile §4c wording with §7 ? so a reader does not believe quantization is active. If P3 real fixtures land before quantization, gate that on the quantization being implemented, since at small n the raw 29.97/59.94/4K-width values meaningfully narrow the source camera."

}

],

"coverageGaps": [

"Could not run the cross-OS CI matrix or numpy-bearing full suite in this dev container (no numpy/torch/cv2) ? relied on the stated 759 pass / 80.49% / cross-OS-green as given; verified only pure-stdlib logic paths locally.",

"Did not audit git HISTORY of eval\_baselines/ for previously-committed real names (P0 in the doc is still ?/open) ? scope here was the 4 merged feature PRs, not the pre-existing leak remediation.",

"The off-git surrogate?source map is by definition not in the repo, so its real-world handling (does the owner actually keep it off-git, is it rotated) is unverifiable from code ? confirmed only that the code writes NO map and the slug is non-reversible.",

"No real `cleared\_real` fixture exists yet (P3 is ?), so the findings are about what the tooling WOULD commit when the owner runs --freeze-real on real data, validated via synthetic stand-ins, not an actual committed real fixture."

],

"verdict": "NOT CLEAN ? two high-severity privacy gaps the per-PR verification missed. The refusal gate, the os.urandom surrogate non-reversibility, the no-on-git-map property, and the truthy-bypass resistance are all genuinely sound. But `\_scan\_forbidden` is incomplete (free-text `source\_note`/`label` identifiers are committed unscanned into provenance.json), and the headline minimization claim is contradicted by the code: the full per-frame `(x,y)` shuttle trajectory ? the exact de Montjoye fingerprint ? is committed verbatim for real fixtures, directly against the doc's §4c 'never commit the per-frame CSV' and TM5's 'only ~5 scalars/window'. The owner's 2026-06-16 risk acceptance was made against a doc that overstates the de-attribution actually implemented. Recommend: (a) reconcile the raw-trajectory reality with §4c/§5/TM5 (and re-confirm owner acceptance on the true basis) before any P3 real commit, (b) enforce or drop the unscanned `source\_note`/`label` free-text fields, and (c) add cleanup on the post-write refusal. None of these block the SYNTHETIC fixtures shipped today (they carry no real data), but they DO block the P3 real-data path the refusal gate exists to enable."

},

{

"lens": "CORRECTNESS + COVERAGE",

"summary": "Audited the shipped Golden Regression Fixtures harness (PRs #186/#189/#191/#192, worktree HEAD 854db56, one DOC commit past 8d3e185) by reading the core module + tests + fixtures + CI + docs and empirically exercising every branch the prompt flagged. The feature is fundamentally sound: candidate reordering IS caught (positional compare), gt\_hash/shuttle-key/candidate-count/segment-count drift IS caught, the refusal gate blocks all unsafe combos,

the time-origin reset is genuinely metric-invariant, and cross-OS determinism is real (strict >1e-6 absorbs a single-ulp libm flip; -0.0 normalized; stable sort + dframes>0 guard make dup frames deterministic). But the per-PR verification missed five concrete gaps, two of which are real correctness/privacy holes I reproduced by running code: (1) compare\_expected silently ignores an EXTRA field present only on one side of score/top-level; (2) \_scan\_forbidden's schema-literal skip can MISS a real source-path leak whose token is a substring of a schema literal (traj/label/expected/prov/?). Plus a doc over-claim (the replay path is NOT numpy-free), an unguarded provenance schema-version path in load\_fixture, and the fact that the characterization gate's entire MISMATCH-detection arm and the privacy scan's two core branches have NO negative test in the committed suite ? they are only ever asserted to return []/not-raise on the happy path.",

```
"findings": [
  {
    "title": "compare_expected silently ignores an EXTRA field present on only one side
(score block + top-level keys)",
    "severity": "medium",
    "evidence": "backend/eval/golden_fixtures.py:823-830 iterates a FIXED list (int_fields +
float_fields) and reads rsc.get(k)/csc.get(k); a key present in only one `score` dict but not in
those lists is never compared. Verified: injecting score['bogus']=99 into the recomputed side
returns `[]` (no mismatch). Likewise the top-level comparison (793-796) checks only
slug/schema_version/gt_hash/iou by name ? any NEW top-level key in either side is invisible. By
contrast the shuttle block (806) DOES use `set(rs)!=set(cs)` and candidates/control_segments use
count checks, so those are guarded ? the asymmetry is the bug. A future schema change (or a
regression that adds a stray/leaked key into the score/top-level block of expected.json) would
pass the characterization gate green, defeating its 'catches ANY behaviour change even at
constant F1' claim (docs §4d).",
    "recommendation": "Add a key-set equality assertion for the `score` dict (mirror the
shuttle block's `set(rsc)!=set(csc)` ? exact mismatch) and for the top-level keys, BEFORE the
per-field loops. Then add a negative test asserting compare_expected returns a non-empty list
for an extra/missing field.",
  },
  {
    "title": "_scan_forbidden schema-literal skip can MISS a real source-path leak (false
negative in the privacy gate)",
    "severity": "medium",
    "evidence": "backend/eval/golden_fixtures.py:530 skips any source-path token `t` where
`any(t in lit for lit in _SCHEMA_FILENAME_LITERALS)` ? i.e. the token is a SUBSTRING of a schema
literal. The intent (comment 528-529) is to stop a short stem like `traj` ? `trajectory.csv`
from false-positiving, but it over-skips: tokens
`traj`,`trajectory`,`label`,`labels`,`expected`,`expect`,`prov`,`provenance`,`exp`,`oven`,`en`
are all substrings of the literals and get skipped. Verified: source path `/secret/label.csv`
(stem `label`) + written text containing `label arena` ? NO RefusalError raised, the leak is
MISSED. A real owner source literally named `label.csv`/`trajectory.csv`/`expected.json`
(plausible ? TrackNet writes `_predict.csv` but users rename) leaking its stem via
`--source-note`/`--label` would not be caught. The check direction is inverted from what safety
wants.",
    "recommendation": "Only skip when the token EQUALS a schema-filename literal (`t in
_SCHEMA_FILENAME_LITERALS` / exact match), not when it is a substring of one. Better: never let
`--source-note`/`--label` flow into the written text unscrubbed, and add negative tests for
stems like `label`/`traj`/`expected`.",
  },
  {
    "title": "Over-claim: the replay/CONTROL path is documented as 'pure-stdlib / no numpy'
but numpy is a hard import-time dependency",
    "severity": "low",
    "evidence": "Module docstring backend/eval/golden_fixtures.py:8-9 ('keeps all candidate
intervals ? uses no numpy') and 34-35 ('pulls in no numpy/cv2 on the CONTROL path'), plus
eval_baselines/fixtures/README.md:30-33 ('no numpy / logistic-regression'), claim the path is
numpy-free. Verified by blocking numpy: `import golden_fixtures` fails with ImportError ? the
chain is golden_fixtures ? experiment.py:49 `from backend.eval.classifier import
LogisticRegression` ? classifier.py:18 `import numpy as np`. So numpy is required at IMPORT
time. The CI comment in ci.yml:117 is itself self-contradictory: 'pure-stdlib (only numpy, no
cv2/torch)'. The accurate, already-correct nuance (per the pre-verified note) is that CONTROL
avoids numpy EXECUTION (the BLAS-nondeterministic LogisticRegression math), which is what
preserves D5 determinism ? not that it avoids numpy import. The golden-crossos CI job installs
```

```

numpy precisely because import needs it.",
    "recommendation": "Soften the prose to 'no numpy EXECUTION on the CONTROL path (numpy is
imported transitively via experiment?classifier but never run, so no BLAS nondeterminism)'. Fix
the contradictory ci.yml:117 comment. No code change needed; determinism is intact."
},
{
    "title": "load_fixture schema-version guard does not protect against a forward-
incompatible provenance.json",
    "severity": "low",
    "evidence": "backend/eval/golden_fixtures.py:761-764 computes `sv =
expected.get('schema_version', prov.get('schema_version', 0))` and raises only on that single
value. Verified CASE C: expected.json with schema_version=1 (current) but provenance.json with
schema_version=99 ? load_fixture does NOT raise; the future-version provenance is read and used
silently. The dedicated test tests/test_golden_fixtures.py:44-48
(`test_schema_version_le_current`) checks BOTH files are <= current, but that asserts a property
of the committed corpus ? it does not exercise load_fixture's guard, and the production loader's
guard only covers expected.json (with prov as a fallback only when expected lacks the field).",
    "recommendation": "In load_fixture, check max(expected_sv, prov_sv) > CURRENT_SCHEMA
(guard both independently), and add a negative test that a future provenance schema_version
raises."
},
{
    "title": "Characterization gate's mismatch-detection arm and the privacy scan's two core
branches have NO negative test in the committed suite",
    "severity": "medium",
    "evidence": "Coverage of backend/eval/golden_fixtures.py from
tests/test_golden_fixtures.py + tests/test_golden_real_fixtures.py is 85% (60 lines missed).
Every committed call to compare_expected asserts it returns `[]` (happy path:
test_golden_fixtures.py:37 via the perturbation test which actually compares control_segments
with raw `!=` not compare_expected; :121 and test_golden_real_fixtures.py:226 assert == []). NO
test asserts compare_expected returns a NON-EMPTY list for a regression ? the gate's entire
raison d'être (lines 784,789-791,800,807,817) is unexercised. Likewise _scan_forbidden's
FORBIDDEN_KEYS branch (513) and MD5 branch (519) are uncovered: only the source-path leak via
source_note is tested (test_golden_real_fixtures.py:155); there is no test that a
person/audio/pose key or a 32-hex MD5 in the written output triggers RefusalError.
refresh_snapshot (452-461, incl. --all and the no-args ValueError) is entirely untested. The
whole CLI main() (947-986) is untested except via the single --check happy path.",
    "recommendation": "Add negative tests: (a) compare_expected returns non-empty for a
changed float beyond 1e-6, a changed int, a candidate count/reorder, and an extra/missing key;
(b) _scan_forbidden raises for an injected FORBIDDEN_KEY and for an injected MD5 string; (c)
refresh_snapshot --all round-trips and the no-target call raises. These lock the gate's failure
semantics, not just its pass semantics."
}
],
"coverageGaps": [
    "compare_expected: NO test asserts a non-empty mismatch list (regression-detection arm
untested) ? lines 784/789-791/800/807/817 uncovered. The perturbation test compares
control_segments with a raw `!=`, bypassing compare_expected's logic.",
    "_scan_forbidden: FORBIDDEN_KEYS branch (513) and MD5 branch (519) have no negative test ?
only the source-path/source_note leak path is exercised. The person/audio/pose/MD5 leak refusals
are unverified by the suite.",
    "refresh_snapshot (452-461) entirely untested, including --all and the 'needs --slug or
--all' ValueError; CLI main() refresh/freeze-synthetic/freeze-real branches (947-986) untested
except the --check happy path.",
    "load_fixture's >CURRENT_SCHEMA raise (763) not exercised by a committed test (test only
asserts <=current on the committed corpus, never that a future version raises); provenance-only
forward-incompat is unguarded.",
    "CI: test_golden_real_fixtures.py (refusal gate + de-attribution + privacy scan) runs ONLY
in the ubuntu full-suite ? the cross-OS golden-crossos job runs only --check +
test_golden_fixtures.py. The freeze-real tool is intended to run owner-side (often Windows) but
its privacy/refusal logic has no Windows/macOS CI coverage (acceptable since it's OS-independent
stdlib, but undocumented).",
    "No fixture exercises a ZERO-candidate or empty-trajectory case in the committed corpus
(only verified ad-hoc); the 3 synthetic slugs all yield clean 1-2 candidate, F1=1.0 cases, so
degenerate-shape regressions (empty candidates list serialization, score with num_pred=0) are

```

not snapshot-locked."

```
  ],
  "verdict": "NOT CLEAN ? the feature is well-engineered and the headline guarantees hold
(reordering/gt_hash/count drift caught, refusal gate sound, time-origin reset provably metric-
invariant, cross-OS determinism real), but the per-PR verification missed 5 concrete gaps. Two
are real correctness/privacy holes reproduced by running code: compare_expected silently ignores
an EXTRA score/top-level field (weakens the 'catches ANY change' claim), and _scan_forbidden's
schema-literal skip can MISS a genuine source-path leak (false negative in the privacy gate).
The other three are a doc over-claim (replay is NOT numpy-free ? numpy is an import-time dep),
an unguarded provenance schema-version path in load_fixture, and a notable test-coverage
asymmetry: the characterization gate's mismatch-detection arm and the privacy scan's
FORBIDDEN_KEYS/MD5 branches are only ever tested on the happy path, never asserted to fire on a
regression/leak. The 70% global coverage floor is sustainable mechanically (module sits at 85%
and the suite grows by manifest accretion), but coverage % is the wrong lens here ? the gate's
FAILURE semantics, not its line count, are what's under-tested, and that is exactly where these
gaps live. Recommend: fix the two correctness holes (key-set equality in compare_expected's
score/top-level; exact-match not substring in the scan skip), correct the numpy prose, guard
prov schema_version, and add the negative tests enumerated above before relying on this as a
regression tripwire."
```

```
  },
  {
    "lens": "ROBUSTNESS to the concurrent quality track + future windowing/gate change (audit of
shipped Golden Regression Fixtures, PRs #186/#189/#191/#192, worktree gf-doc @ 8d3e185)",
    "summary": "The harness is well-engineered for its stated CONTROL/Tier-0 job and the per-PR
verification (759 pass, 80.49%, cross-OS green, CONTROL avoids numpy execution) is accurate.
Against the ROBUSTNESS lens the picture is mixed-but-mostly-safe: the most-feared interactions
fail LOUDLY, not silently. If
`distill_local.build_candidates`/`trajectory_to_action_windows`/`rally_gate` SIGNATURES or float
MATH change, the replay either raises (TypeError on the 3-tuple call; module-level
AssertionError on FEATURE_NAMES drift) or the parse-compare diffs loudly ? exactly the desired
behaviour, and net_crossings is exact-int compared so a crossing-count change is caught even
though CONTROL gates at min_crossings=0. The pinned-literal-vs-SERVED divergence is real and is
the intended safety mechanism (a SERVED retune cannot churn the 3 pinned knobs). HOWEVER there
is one genuine blind spot that the per-PR review missed: the W1 bounded-windowing dimension
(max_window_duration, PR #187 ? the very windowing session the lens names) is pinned in
provenance/manifest as max_win=0.0 but is STRUCTURALLY UNREACHABLE from the replay, because
`pinned_proposal` returns only `(vt,mg,mwd)` and `build_candidates` has no max_window param. So
if a future PR flips SERVED's max_window_duration>0 or changes the `_split_overlong_groups`
algorithm, production windowing changes while these fixtures stay GREEN (silent pass).
Separately, docs/GOLDEN_REGRESSION_FIXTURES.md §4a over-claims the reuse map (says the replay
goes through windowing.build_windows / WindowingPreset / "default SERVED" ? it does NOT;
windowing.py is not on the replay path at all). Net-crossings (#188/#190 net-crossings gate)
does interact with the CONTROL path only as a stored feature, not as a segment filter (CONTROL
min_crossings=0), which is correctly characterized. The control path itself is safe. Findings
below; none are correctness-breaking for what is committed today, but the max_win gap + doc
overclaim should be fixed before P3 real fixtures or any W1 promotion lands.",
```

```
    "findings": [
      {
        "title": "W1 bounded-windowing (max_window_duration) is pinned-but-IGNORED in the replay
? a future SERVED W1 flip or split-algorithm change passes the fixtures SILENTLY",
        "severity": "medium",
        "evidence": "golden_fixtures.py:75 pins PINNED_MAX_WINDOW=0.0 and writes it into
provenance.windowing.max_win (lines 386-391) and manifest (728-730); the committed fixtures +
manifest.json all carry \"max_win\": 0.0. But the replay never threads it: `_pinned_proposal`
(golden_fixtures.py:287-293) returns only the 3-tuple (vt,mg,mwd), and
`distill_local.build_candidates` (distill_local.py:52-59) has NO max_window_duration parameter ?
it calls trajectory_to_action_windows with only velocity_thresh/merge_gap/min_window_duration,
so max_window_duration defaults to 0.0 (tracknet_runner.py:265) regardless of what provenance
says. The W1 split logic (tracknet_runner.py:334-369 `_split_overlong_groups`, the PR #187 over-
merge fix) is therefore unreachable from the harness. IndexingConfig.max_window_duration default
is currently 0.0 (config/models.py:138) so today there is no divergence, but the SERVED preset
DOES source max_window_duration from that config (windowing.py:99). If a future PR sets that
default >0 (or retunes min_split_gap, or changes the split algorithm), the production/SERVED
windowing changes while every golden fixture stays GREEN ? a silent miss of exactly the
windowing dimension the lens names.",
```



```

    "recommendation": "Either (a) thread max_win through the replay: extend
`_pinned_proposal` to also return max_win and add a max_window_duration parameter to
build_candidates (or replay via windowing.build_windows(preset) as the doc claims), so the
pinned value is actually consumed and a W1 change is characterized loudly; or (b) if
intentionally keeping W1 out of Tier-0, drop max_win from the pinned schema and DOCUMENT in the
module/README that W1 bounded-windowing is deliberately uncharacterized ? do not pin a knob the
replay ignores, which currently reads as coverage that does not exist. Add a regression test
that flips a fixture's provenance max_win and asserts the replay segments change (mirroring
test_perturbing_trajectory_changes_segments), proving the knob is live before relying on it for
the W1 promotion."
},
{
    "title": "Doc §4a over-claims the replay reuse map: it names windowing.build_windows /
WindowingPreset / 'default SERVED', none of which the replay touches",
    "severity": "low",
    "evidence": "docs/GOLDEN_REGRESSION_FIXTURES.md:79-81 states the single replay path is
`parse_trajectory_csv ? windowing.build_windows(preset) ? distill_local.build_candidates ?
rally_gate net_crossings ? experiment.predict(CONTROL) ? merge_overlaps ? metrics.score` and
'Reuses verbatim: ... windowing.WindowingPreset (default SERVED)'. The shipped code never
imports windowing.py: golden_fixtures.py imports distill_local, experiment, gt_loader, metrics,
regression, tracknet_runner (lines 49-55) and calls distill_local.build_candidates with a raw
3-tuple (line 319). `grep build_windows|WindowingPreset|windowing.SERVED` in golden_fixtures.py
matches only docstring prose, never a call. The module docstring (golden_fixtures.py:11-16)
gives the CORRECT map (no windowing.build_windows); the doc is the stale one.",
    "recommendation": "Fix §4a to match the shipped path (drop windowing.build_windows /
WindowingPreset / 'default SERVED' from the reuse list, since the harness deliberately pins
literals NOT the named preset ? which the doc itself argues for in §4c). This also removes a
contradiction: §4a says 'default SERVED' while §4c/README correctly say the design avoids the
SERVED constant. Cheap to fix; matters because the doc is the cold-start map for the next
contributor reasoning about what a windowing change will/won't catch."
},
{
    "title": "Signature/algorithm changes to the reused stack DO fail loudly ? confirmed
robust (positive finding)",
    "severity": "low",
    "evidence": "FEATURE_NAMES drift is caught at IMPORT: golden_fixtures.py:82-84 `assert
tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES` raises at module load, breaking
collection loudly. A build_candidates `proposal` shape change (e.g. 3-tuple?4-tuple or
?WindowingPreset) breaks the explicit call at line 319-320 with TypeError/unpack error. A
trajectory_to_action_windows MATH change (velocity formula tracknet_runner.py:301-304, merge
logic 324-332) shifts floats ? parse-compare diffs (compare_expected,
golden_fixtures.py:803-813, approx 1e-6). A rally_gate crossing-count change (estimate_play_axis
/ count_net_crossings) is caught because net_crossings is compared EXACT-int
(compare_expected:810-811), even though CONTROL gates at min_crossings=0 so it does not affect
control_segments. A predict(CONTROL) merge change shifts control_segments (compared approx,
815-821). test_perturbing_trajectory_changes_segments (test_golden_fixtures.py:86-109) proves
re-derivation, not echo.",
    "recommendation": "No change needed ? this is the robustness the feature was built for
and it holds. Worth stating explicitly in the audit so the team knows the loud-fail guarantee is
real for the (vt,mg,mwd) windowing knobs, build_candidates/rally_gate signatures, and the
CONTROL decision path. The ONLY uncovered axis is W1/max_win (separate finding)."
},
{
    "title": "schema_version guard only rejects FORWARD-incompatible (sv>CURRENT); an
OLDER/equal committed fixture against a bumped CURRENT_SCHEMA is replayed against new code with
no migration",
    "severity": "low",
    "evidence": "load_fixture (golden_fixtures.py:761-764) raises only when `sv >
CURRENT_SCHEMA`. When the schema is later bumped (CURRENT_SCHEMA=2) but an old committed fixture
still carries schema_version=1, it is accepted and replayed by the v2 code. replay_fixture
always STAMPS the recomputed snapshot with the current CURRENT_SCHEMA (line 343), while
compare_expected does an EXACT compare of schema_version (line 794) against the committed
(older) value ? so a stale fixture would fail on the schema_version field with a confusing '2 !=
1 (exact)' rather than a clear 'fixture needs re-freeze for schema v2' message. This is a
forward-looking corpus-growth concern (the deferred M4 quality-floor + P3 real fixtures will

```

```

accrete fixtures over time across schema bumps).",
  "recommendation": "When CURRENT_SCHEMA is first bumped, add an explicit lower-
bound/upgrade path: either a per-fixture migration or a clear message in
load_fixture/compare_expected that an older schema_version requires --refresh-snapshot (with the
$6 rubber-stamp caveat). Not urgent at schema v1 with 3 fixtures; flag it so the bump does not
surprise. Consider a test that freezes at v1, bumps CURRENT_SCHEMA, and asserts a clean
actionable error."
},
{
  "title": "P3 real-fixture runtime / corpus-growth: --check and the cross-OS job replay
EVERY manifest fixture serially with no cap, and freeze_real_fixture's frame_width is
unvalidated against the trajectory",
  "severity": "low",
  "evidence": "_cmd_check (golden_fixtures.py:879-902) and
test_replay_matches_committed_expected (test_golden_fixtures.py:27-40, parametrized over the
whole manifest) replay all fixtures with no row-count cap or per-fixture size bound ? the red-
team item 'row-count cap ? fold into M4' (doc $7 M3 row) is still ?. As P3 commits real de-
attributed fixtures and the corpus grows, the golden-crossos job (ci.yml:105-122, 3 OSes)
runtime grows linearly with no guard. Separately, freeze_real_fixture takes fps/frame_width as
caller-supplied scalars (golden_fixtures.py:566-567) used both to compute t0_sec for the metric-
invariant reset (624-628) and written to provenance (646-647); nothing checks frame_width
against the actual trajectory X range, so a wrong --frame-width silently changes the normalized
velocity ? the frozen snapshot, with no loud signal (the reset stays metric-invariant per
test_time_origin_reset_is_metric_invariant, but the absolute feature values bake in the
operator's possibly-wrong fw).",
  "recommendation": "For M4/P3: add the deferred row-count / total-fixture cap (or a fast-
subset vs full-corpus split in CI) before the corpus grows, and consider sharding the cross-OS
matrix job if runtime climbs. For freeze_real_fixture, add a sanity check that frame_width is
within a plausible factor of max observed trajectory X (warn or refuse on gross mismatch) so an
operator typo in --frame-width is caught at freeze time rather than baked into a committed real
fixture. Both are forward-looking, not blocking today's synthetic-only commit."
}
],
"coverageGaps": [
  "Could not run the suite/CLI in this worktree (the lens-verified 'numpy installed, no GPU
stack' env was the per-PR machine; this dev container lacks numpy/torch/cv2 per CLAUDE.md), so
the cross-OS float-stability claim and the exact replay outputs were read from committed
expected.json, not re-executed here. The 1e-6 tolerance vs cross-CPU math.sin/sqrt drift was
reasoned about, not re-measured on a third OS.",
  "Did not audit P0 (the live name-leak remediation in
eval_baselines/heuristic_lovo_n6.json, gemini_wrapper_2026-06-10.json, eval_partial_labels.py) ?
out of scope for these 4 PRs (P0 is ? / separate), but note the CI leak-scan that doc $7 ?/P0
calls a REQUIRED status check is NOT present in ci.yml (the golden-crossos job runs --check +
the fixtures test, not a tree-wide MD5/name scan); the positive _scan_forbidden only runs inside
freeze_real_fixture, not over committed fixtures in CI.",
  "Did not verify the EDPS-v-SRB / de-attribution LEGAL claims ($3) ? flagged as owner-
accepted researched-risk, not a code-auditable property.",
  "The M4 quality-floor gate and its windowing-fingerprinted regression_baseline.json are ?
(not shipped), so the 'quality-floor catches a quality drop' half of the doc's two-gate design
is not yet auditable; only the characterization gate is live."
],
"verdict": "SHIP-SAFE for the committed synthetic Tier-0, with two should-fix items before
the quality track / P3 real fixtures lean on it. The control path and the loud-fail guarantees
against signature/math/net-crossing changes are genuinely robust ? the per-PR verification was
sound for what it claimed. The per-PR review MISSED one real robustness gap (medium):
max_window_duration / W1 bounded-windowing (PR #187, the named windowing session) is pinned in
provenance+manifest as max_win=0.0 but is structurally unreachable from the replay
(build_candidates takes no such param), so a future SERVED W1 flip or split-algorithm change
passes the fixtures SILENTLY ? the one place the lens's 'pass silently (bad)' fear is realized.
Plus a low-severity doc overclaim ($4a names windowing.build_windows/WindowingPreset/'default
SERVED' on a path that uses none of them). Neither breaks anything committed today; fix the
max_win pinned-but-ignored gap (thread it through or stop pinning it) and the $4a reuse-map
before W1 promotion or real fixtures land."
},
{

```

```

    "lens": "HONESTY / over-claim",
    "summary": "Audited the shipped Golden Regression Fixtures feature (PRs #186/#189/#191/#192)
at master HEAD 8d3e185 in the gf-doc worktree, through the honesty/over-claim lens. The core
honesty discipline is genuinely good: the $6 \"behaviour-locking NOT correctness\" +
\"synthetic-tier = plumbing not field quality\" caveats DO ride in the primary characterization-
test assertion message (tests/test_golden_fixtures.py:38) and in both the --check FAIL
(golden_fixtures.py:892-893) and summary (golden_fixtures.py:900-901) lines, not only docs; the
\"owner-accepted researched-risk, NOT retained-counsel\" framing is honest and consistent across
the header, $3, $8, P2 row, and $10; the time-origin-reset metric-invariance claim is real and
tested; CONTROL avoids numpy execution; the cross-OS 1e-6 parse-tolerance guard correctly covers
the committed synthetic fixtures. But the per-PR verification missed several real gaps: a
docstring/tracker over-claim that the --refresh CLI \"requires\" a reason (it only warns), a
\"positive privacy assertion\" that does not actually catch the identifiers it implies it does,
a cross-OS-gated test that will break the suite the moment the shipped P1 tool's own output (a
cleared_real fixture) is committed, and a doc $4c \"never commit the per-frame CSV\" rule the
shipped freeze_real_fixture violates by design.",
    "findings": [
        {
            "title": "refresh_snapshot docstring + tracker claim the CLI 'requires' a reason; it
does not ? only a soft warning, exit 0",
            "severity": "medium",
            "evidence": "backend/eval/golden_fixtures.py:451 docstring says the rubber-stamp
mitigation is '...recording a reason (the CLI requires one)'. The doc $6
(GOLDEN_REGRESSION_FIXTURES.md:134) says 'Require a recorded reason' and $4 implies enforcement.
But main() at golden_fixtures.py:955-960 does 'if not args.reason:' -> prints a WARNING to
stderr -> still calls refresh_snapshot() and returns 0. No --reason is required; the guardrail
the docstring advertises is unenforced. This is the exact gate that prevents a real regression
from being silently rubber-stamped, so the over-claim matters.",
            "recommendation": "Either make --reason mandatory for --refresh-snapshot (ap.error if
absent, like the --freeze-real required-args check at :964-969), or correct the docstring/doc to
'the CLI WARNS if one is missing' so the stated guardrail matches the code."
        },
        {
            "title": "'Positive privacy assertion' over-claims: it catches MD5 / 6 forbidden
substrings / source-path tokens, NOT arbitrary names, match titles, or dates in
--label/--source-note",
            "severity": "medium",
            "evidence": "_scan_forbidden (golden_fixtures.py:501-535) scans written text only for
FORBIDDEN_KEYS substrings (person/pose/gait/face/audio/player, :88), a 32-hex MD5 (:95), and
source-path tokens (:522-535). The docstring calls it a 'POSITIVE privacy assertion ($4c, red-
team?): raise if text leaks anything attributable' and doc $4c 'Metadata strip ? no filename,
video_id, match/event/player identity, capture date'. But a human name ('Lin Dan'), match title,
club, or capture date placed in --label or --source-note is NOT detected ? it passes silently.
The only test of this path, test_positive_privacy_assertion_catches_leak_in_source_note
(tests/test_golden_real_fixtures.py:155-164), exercises ONLY the source-PATH case
(source_note=str(traj)); there is no test (and no mechanism) for a free-text name/date leak.",
            "recommendation": "Tighten the docstring/$4c to state precisely what the scan
mechanically guarantees (MD5 + forbidden-stream substrings + source-path tokens) and that
arbitrary identifiers in operator-supplied free text (--label/--source-note) are the OPERATOR's
responsibility, not machine-caught. Optionally add a date-pattern/digit-run heuristic and
require --label/--source-note to match a conservative allowlist."
        },
        {
            "title": "Cross-OS-gated test_synthetic_provenance_tags hard-rejects the shipped P1
tool's own output (kind='cleared_real'), so the suite breaks the moment P3 commits a real
fixture",
            "severity": "medium",
            "evidence": "tests/test_golden_fixtures.py:69-77 is @parametrize over ALL manifest SLUGS
and asserts prov['kind']=='synthetic' and prov['license']=='synthetic'. freeze_real_fixture
writes kind='cleared_real', license=<source license> and _append_manifest_entry adds it to the
SAME manifest (golden_fixtures.py:639-714). I confirmed empirically: freezing a cleared_real
fixture then applying the assertion yields 'would FAIL test_synthetic_provenance_tags (kind is
cleared_real)'. This test is in the golden-crossos CI job (ci.yml:122), so committing the very
fixtures P1 was built to produce (P3, the tracker's stated next step) will red the cross-OS
suite. The per-PR verification could not catch this because no real fixture is committed yet (P3

```

```

?)."
    "recommendation": "Split the assertion by kind: keep synthetic-tag checks for
kind=='synthetic' slugs, and add a parallel cleared_real-tag check (kind=='cleared_real', owned
True, minors_free True, non-blank license) ? mirroring the split already present between
test_golden_fixtures.py and test_golden_real_fixtures.py. Rename the test to reflect it gates
all Tier-0 fixtures, not only synthetic."
    },
    {
        "title": "Doc §4c 'never commit the per-frame (Frame,Visibility,X,Y) CSV' contradicts
the shipped freeze_real_fixture, which writes the full per-frame trajectory.csv into the
committable fixture dir",
        "severity": "medium",
        "evidence": "GOLDEN_REGRESSION_FIXTURES.md §4c (:96) and §4b Tier-0 contents (:86) state
Tier-0 keeps only '~5 aggregate shuttle scalars/window' and 'Raw-trajectory exclusion ? never
commit the per-frame CSV'. But freeze_real_fixture writes the de-attributed but still PER-FRAME
trajectory rows to fdir/'trajectory.csv' (golden_fixtures.py:635) and registers its path in the
manifest/provenance (:700, :658). The synthetic fixtures do likewise
(eval_baselines/fixtures/*/trajectory.csv exists). So the shipped Tier-0 fixture DOES commit the
per-frame trajectory, directly contradicting the design rule the doc presents as a non-
negotiable anti-attribution measure. The de Montjoye/EDPS re-ID argument in §3 (:70) hinges on
exactly this per-frame fingerprint.",
        "recommendation": "Reconcile honestly: either (a) update §4c/§4b to state the per-frame
trajectory.csv IS committed (de-attributed via time-origin reset + random slug + metadata strip)
and re-justify the re-ID posture with the per-frame data in scope, or (b) change the freeze to
persist only the aggregate window/feature snapshot needed for replay. As written, the doc claims
a minimization that the code does not perform."
    },
    {
        "title": "Per-fixture '[ok] ... F1=1.000000' CLI line surfaces synthetic F1=1.0 without
the 'not field quality' caveat on that line",
        "severity": "low",
        "evidence": "_cmd_check prints '[ok] {slug}: ... F1={sc['f1']:.6f}` per fixture
(golden_fixtures.py:898-899) ? 'F1=1.000000'. The 'synthetic-tier = plumbing correctness, not
field quality ? §6' caveat is only on the trailing summary line (:900-901). A reader scanning
per-fixture output (or copying a single line) sees a perfect F1 with no inline disclaimer. The
committed expected.json files also carry f1:1.0 with no inline caveat (the caveat lives in
provenance.note / README). §6 (:128) and the DoD (:193) insist the caveat 'must ride in the test
pass/fail message'.",
        "recommendation": "Append a short '(synthetic plumbing, not quality)' tag to the per-
fixture [ok] line, or drop the F1 number from the per-fixture line and report it only under the
caveated summary. Low severity because the summary line and surrounding docs do carry the
caveat."
    },
    {
        "title": "'cross-OS enforced' wording slightly overstates given the doc's own §9 lists
'required status check' as an unchecked TODO",
        "severity": "low",
        "evidence": "Progress log (:163) says the golden-crossos job is green on 3 OSes ?
'cross-OS **enforced**'. The job (ci.yml:105-122) does run on every PR across
ubuntu/windows/macOS, but 'enforced' implies it blocks merge, which depends on GitHub branch-
protection (a required status check). §9 DoD (:189) still lists making the CI leak-scan 'a
required status check' as unchecked, and there is no claim that golden-crossos is branch-
protected. fail-fast:false (ci.yml:108) also means one OS failing doesn't fail the others. So
'enforced' is aspirational vs 'runs on every PR'.",
        "recommendation": "Softens to 'runs cross-OS on every PR (3-OS matrix)'; reserve
'enforced' for when the job is configured as a required status check, consistent with how §9
treats the leak-scan."
    }
],
"coverageGaps": [
    "GitHub branch-protection / required-status-check configuration is server-side and could
not be verified from the repo ? the 'enforced' claim's truth depends on it.",
    "test_golden_real_fixtures.py and the P1 freeze_real / refusal-gate / privacy-scan code
paths are NOT exercised in the golden-crossos cross-OS CI job (only test_golden_fixtures.py +
--check run there); their cross-OS determinism is unverified by CI, though the shared float math

```

and 1e-6 tolerance make a regression unlikely.",

"Did not independently re-run the full 759-test suite / coverage / ruff / mypy (per instructions these were pre-verified); findings here are static + targeted-dynamic (ran --check and the two golden test files locally on Python 3.13/Windows, plus a freeze\_real\_fixture simulation).",

"Could not assess whether a real Fork-A source actually satisfies the §3 conditions (no real data is committed; P3 ?) ? only the tooling and its gate were auditable.",

"The legal/jurisdictional claims in §3 were read for internal consistency/labeling honesty, not evaluated for legal correctness (out of scope and explicitly disclaimed as 'not legal advice')."

],

"verdict": "NOT CLEAN ? the feature's headline honesty caveats (§6 behaviour-locking, synthetic?field-quality, owner-accepted-not-counsel) are genuinely well carried into code/test messages, but the audit surfaced six real over-claim/gap items the per-PR verification missed. Two are forward-facing correctness traps that will bite when P3 (the shipped tool's intended use) lands: test\_synthetic\_provenance\_tags will red the cross-OS suite on any cleared\_real\_fixture (medium), and the doc's 'never commit the per-frame CSV' rule is already violated by the shipped freeze\_real\_fixture (medium). Two are guardrail/claim mismatches: refresh\_snapshot's 'CLI requires a reason' is unenforced (medium), and the 'positive privacy assertion' does not catch arbitrary names/dates it implies it does (medium). Two are low-severity wording over-reaches ('[ok] F1=1.000000' uncaveated per-line; 'cross-OS enforced'). No critical/high privacy or determinism hole found; recommend fixing the two medium forward-traps before any P3 real\_fixture commit and correcting the two medium claim/doc mismatches."

}

]

VERIFICATIONS (JSON):

```
[
{
  "claim": "The time-origin reset in golden_fixtures.freeze_real_fixture is NOT metric-invariant for non-whole-second t0, contradicting the doc/code \"provably metric-invariant / bit-identical scores\" claim, with the worst delta exceeding _FLOAT_TOL=1e-6.",
  "verdict": "confirmed",
  "reasoning": "Both factual halves reproduce exactly against the worktree code (HEAD 854db56).\n\n(1) Non-invariance: I ran the finding's own sweep ? freeze the same synthetic source via freeze_real_fixture with time_origin_reset=True vs False, frame offsets 0..39 at fps=30. Score equality broke on EXACTLY the 26/40 offsets the finding names ([1,2,4,5,7,8,...,38]). Mechanism (traced at offset=7): the source labels.csv is itself written at 6dp (start=0.233333). WITH reset, the full-precision t0_sec=7/30 is subtracted, cleanly cancelling to 0.000000; WITHOUT reset, the already-rounded 0.233333 (off the true 0.2333... by ~3.3e-7) flows through, shifting mean_start_error across the 6dp boundary: 0.033333 (reset) vs 0.033334 (no-reset). This contradicts the \"Provably metric-invariant ... bit-identical scores\" claim at docs/GOLDEN_REGRESSION_FIXTURES.md:94 and the code docstrings at golden_fixtures.py:619-623 and :583-586, plus the test docstring/assertion at tests/test_golden_real_fixtures.py:13-14,189-191.\n\n(2) Past the cliff: the worst measured |delta| is exactly 1.0000000000001e-06 (at offset=1, mean_start_error 0.033333 vs 0.033334), STRICTLY greater than _FLOAT_TOL=1e-6 (golden_fixtures.py:91). compare_expected.approx uses `> _FLOAT_TOL` (golden_fixtures.py:788) ? I confirmed it flags this delta as a mismatch.\n\n(3) Test hides it: the only invariance test (tests/test_golden_real_fixtures.py:172-205) hard-codes offset_frames=9000 ? 9000/30=300.0s exactly (whole-second, t0 exact, reset cancels cleanly). The committed suite passes (12 passed). I reran the test's own `with_reset[\"score\"] == without_reset[\"score\"]` assertion at fractional offsets 7/11/13: all return False (would fail); offset 9000 returns True. So the test passes only because of the whole-second offset, exactly as claimed.\n\nThe proposed fix (round t0_sec to 6dp before subtracting from both streams, or reset by integer frames and derive t0_sec consistently; plus a parametrized fractional-offset test asserting score equality AND |delta|<=_FLOAT_TOL) is correct.\n\nWHY isMustFix=false despite full confirmation: the finding's \"can flip a parse-compare digit/result / high\" framing overstates the operational blast radius. The shipped cross-machine snapshot gate (_cmd_check, golden_fixtures.py:879-902) recomputes the replay from the SAME committed (already-reset) bytes via replay_fixture, which never re-applies the reset ? so it is self-consistent. I verified --check returns [] (clean) even for a fixture frozen at the worst-case offset=7. The non-invariance is purely between the reset vs no-reset *freeze* outputs, not within --check. There are also no committed cleared_real fixtures exercising the reset path, and freeze_real is owner-gated P1 tooling behind counsel sign-off ? not run in CI. So nothing currently relied upon (the green suite, the cross-OS --check gate) is broken or flaky. The
```

```

genuine defects are a documented invariant that is provably false and a test that asserts the
invariant only on the one offset that masks the bug ? a real correctness/coverage issue worth
fixing before the \"metric-invariant de-attribution\" guarantee is depended on, but not an
active suite/gate breakage today.",
    "isMustFix": false
},
{
    "claim": "`_scan_forbidden` does NOT catch free-text identifiers in `source_note`/`label`;
they are committed verbatim into provenance.json, and the only existing test masks this by
passing a path token.",
    "verdict": "confirmed",
    "reasoning": "Verified against the worktree code and proven empirically. `_scan_forbidden`
(backend/eval/golden_fixtures.py:501-535) scans the written JSON text for ONLY three categories:
(a) FORBIDDEN_KEYS biometric substrings (defined at :88 =
\"person\", \"pose\", \"gait\", \"face\", \"audio\", \"player\"), (b) a 32-hex MD5 via _MD5_RE
(:95/:517), and (c) source-PATH tokens ? sp/p.name/p.stem of the trajectory/labels paths
(:522-535). A free-text identifier (player/match/event name) matches none of these.\n\nThe
`label` (golden_fixtures.py:643, default \"cleared_real\") and `source_note` (:663) operator-
supplied fields are written verbatim via str(...) into provenance.json, then scanned at :680-681
? but the scan cannot detect free text. There is NO allow-list, charset restriction, or
controlled-vocabulary check anywhere (grep across the file confirms only str() pass-
through).\n\nEMPIRICALLY PROVEN by running freeze_real_fixture on synthetic input with
label='mens singles final' and source_note='Lin Dan vs Lee, India Open 2024, Court 1': it FROZE
WITH NO REFUSAL and committed both strings verbatim into provenance.json (captured output: label
committed: 'mens singles final'; source_note committed: 'Lin Dan vs Lee, India Open 2024, Court
1').\n\nThe only source_note test (tests/test_golden_real_fixtures.py:155-164) passes
source_note=str(traj) ? a PATH token caught by category (c) ? and its docstring falsely claims
\"source_note is non-identifying by contract ? a path/name in it trips the positive scan.\" No
test exercises a non-path free-text identifier, so the green suite gives false
confidence.\n\nThe doc and CLI assert a contract they do not enforce:
docs/GOLDEN_REGRESSION_FIXTURES.md:93 (\"Metadata strip ? no filename, video_id,
match/event/player identity, capture date\") and CLI help at golden_fixtures.py:936 (\"--source-
note ... NEVER a path/name/id\"). Nothing enforces this for names/ids.\n\nEvery checkable sub-
claim in the finding holds; the proposed fix direction (restrict/drop the fields or move human
notes off-git, plus a test that a free-text identifying note is rejected or never persisted,
plus loud documentation) is appropriate.",
    "isMustFix": true
},
{
    "claim": "Raw per-frame (Frame,Visibility,X,Y) trajectory is committed verbatim for REAL
fixtures via freeze_real_fixture, fully retaining the de Montjoye spatial fingerprint and
contradicting docs/GOLDEN_REGRESSION_FIXTURES.md §4c/§4b/TM5 ? undercutting the owner's
2026-06-16 risk acceptance and the expected.json 5-scalar minimization.",
    "verdict": "partial",
    "reasoning": "The finding's PRIVACY headline is REFUTED, but it surfaces a real DOC-INTERNAL
contradiction (the partial truth).\n\nREFUTED ? the live-leak claim is false:\n1) No
cleared_real fixture exists in the tree. All three committed fixtures (clean_single_rally,
multi_rally_with_deadtime, occlusion_break) are kind=\"synthetic\", license=\"synthetic\" ?
confirmed in eval_baselines/fixtures/manifest.json:5,25,45 and
eval_baselines/fixtures/clean_single_rally/provenance.json:6-9 (\"Synthetic, deterministic,
owned (no real footage)\"). The finding's headline \"committed verbatim for real fixtures\" is
false: zero real fixtures are committed.\n2) The cited evidence is SYNTHETIC, not a real
fingerprint. eval_baselines/fixtures/clean_single_rally/trajectory.csv:2 starts at exactly the
frame-center 640.000000,360.000000 then traces a smooth deterministic parabola ? it is machine-
generated, not extracted from any real person's footage, so the de Montjoye / re-ID argument (4
points -> 95% re-ID) the finding rests on does not apply.\n3) freeze_real_fixture
(backend/eval/golden_fixtures.py:557) is TOOLING ONLY and has not been run on real data:
GOLDEN_REGRESSION_FIXTURES.md:150 (P1) says \"Tooling-only ... no real data committed\"; P3
(line 152, the actual real-data commit) is status Todo, hard-gated by the refusal gate + per-
video owner clearance.\n\nCONFIRMED (the genuine core): freeze_real_fixture DOES write the full
per-frame CSV ? golden_fixtures.py:635 write_trajectory_csv(rows, fdir/\"trajectory.csv\"),
where rows come from _reset_trajectory_rows (lines 538-549) passing X/Y through untouched (only
the frame index shifts). The replay design REQUIRES the committed trajectory.csv as its freeze-
seam input (replay_fixture reads fdir/\"trajectory.csv\" at line 311), and the synthetic
fixtures already commit it (fixtures README line 14 openly lists trajectory.csv as a committed

```

file). This genuinely contradicts the doc's claims that the raw per-frame CSV is excluded: §4c (line 96 \"Raw-trajectory exclusion ? never commit the per-frame (Frame,Visibility,X,Y) CSV\"), §2 D4 (line 45 \"the raw per-frame CSV is never committed (only ~5 aggregate ... scalars/window)\"), and the §4b/TM5 framing that Tier-0 is \"5 shuttle scalars\" / \"only ~5 invariant scalars/window\". expected.json correctly holds only the 5 aggregate scalars (clean\_single\_rally/expected.json:5-11), but the raw CSV sits beside it in the same tracked dir. So the doc is internally inconsistent with its own shipped M1+P1 architecture, and proposed-fix (b) ? reconcile the doc to state the full per-frame trajectory IS committed and re-do the §3/§5 non-attributability analysis on that basis ? is the correct remedy.",

```
    "isMustFix": false
  }
]
```

## 2. assistant (2026-06-16T06:35:55.638Z)

I'll triage these audit findings against their skeptical verifications. Let me first verify the key technical claims by examining the actual code, since the verifications are the authoritative skeptical check on the audit claims.

## 3. user (2026-06-16T06:35:57.579Z)

```
480
481
482     def _new_surrogate_slug() -> str:
483         """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).
484
485         Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
486         random,
487         NON-reversible id with no algebraic link to the source, so it can't be brute-forced
488         over a
489         tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
490         off-git.
491         """
492         return f"fx_{os.urandom(8).hex()}"
493
494     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). Source-
495     path tokens
496     #: that collide with these are NOT a leak ? exclude them so a short source stem like
497     ``traj`` can't
498     #: false-positive against the literal
499     ``trajectory.csv``/``labels.csv``/``expected.json``.
500     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
501         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
502         "provenance", "provenance.json",
503     )
504
505     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
506         """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
507         attributable.
508
509         Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
510         substring,
511         (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
512         path
513         (full path / basename / stem) ? excluding the fixed schema filename literals the
514         writers
515         always emit. The trajectory is shuttle-only by nature; this guards against a
516         *regression*
517         re-introducing a person/audio/pose stream or an id/path back-channel ? never the
518         silent default.
519         """
520         low = text.lower()
521         for key in FORBIDDEN_KEYS:
```

```

512         if key in low:
513             raise RefusalError(
514                 f"{where}: forbidden key substring {key!r} found in written output ? "
515                 f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
516                 f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
517     m = _MD5_RE.search(text)
518     if m:
519         raise RefusalError(
520             f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
521             f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
522     for sp in source_paths:
523         if not sp:
524             continue
525         p = Path(sp)
526         for token in (sp, p.name, p.stem):
527             t = (token or "").strip().lower()
528             # Skip a token that is itself a substring of a schema-literal the writers
always emit
529             # (e.g. a source stem ``traj`` ? ``trajectory.csv``) ? that collision is not
a leak.
530             if not t or any(t in lit for lit in _SCHEMA_FILENAME_LITERALS):
531                 continue
532             if t in low:
533                 raise RefusalError(
534                     f"{where}: source-path token {token!r} found in written output ? the
source "
535                     f"filename/path must never be persisted (§4c metadata strip).")
536
537
538     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
539                               ) -> List[Tuple[int, int, float, float]]:
540         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
``(Frame,Visibility,X,Y)`` rows.
541
542         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
543         pure time shift); only the absolute frame index moves toward 0, severing the match
timeline.
544         """
545         rows: List[Tuple[int, int, float, float]] = []
546         for p in points:
547             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
548         rows.sort(key=lambda r: r[0])
549         return rows
550
551
552     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
553         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant)."""
554         return [(s - t0_sec, e - t0_sec) for s, e in gts]
555
556
557     def freeze_real_fixture(
558         label: str,
559         trajectory_csv_path: Any,
560         labels_csv_path: Any,
561         *,
562         license: str,
563         minors_free: bool,
564         owned: bool,
565         fps: float,
566         frame_width: float,

```



```

567     fixtures_dir: Path = FIXTURES_DIR,
568     slug: Optional[str] = None,
569     time_origin_reset: bool = True,
570     source_note: str = "",
571 ) -> Dict[str, Any]:
572     """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
573     Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. (§4c / §5 / §7
row P1.)
574
575     The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
576     ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string.
577     The committed bytes carry NO filename / video_id / match / player / capture-date ?
only an
578     opaque random surrogate slug, the shuttle-only trajectory (time-origin reset by
default), the
579     reset labels, and a ``kind="cleared_real"`` provenance block. A positive privacy
assertion
580     re-reads the written ``expected.json`` + ``provenance.json`` and refuses if anything
attributable
581     slipped in.
582
583     NOTE (§6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
584     only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
585     attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
586     owner-asserted Fork-A / minors / biometric-exclusion flags.
587
588     Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
589     """
590     # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail (§4c / §7 ?)
-----
591     reasons: List[str] = []
592     if minors_free is not True:
593         reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
594             "of a minor is prohibited ? §3, guardrail 'exclude minors')")
595     if owned is not True:
596         reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
597             "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
598     if not (isinstance(license, str) and license.strip()):
599         reasons.append("license must be a non-blank string (per-record provenance tag ?
§4c "
600             "'provenance hard-fail gate')")
601     if reasons:
602         raise RefusalError(
603             "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
604             "cure provenance/licensing; this gate sits behind counsel sign-off,
§6/P2):\n - "
605             + "\n - ".join(reasons))
606
607     traj_path = Path(str(trajectory_csv_path))
608     labels_path = Path(str(labels_csv_path))
609
610     # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
611     out_slug = slug if slug is not None else _new_surrogate_slug()
612

```

```

613      # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
614      points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
615      if not points:
616          raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
617      gts = load_gt_intervals(str(labels_path), strict=True)
618
619      # --- (d) consistent time-origin reset (default ON; MUST be metric-invariant, §4c)
620      # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
because
621      # metrics.score is built from pure differences (interval_iou, |start-start|, |end-
end|), the
622      # score is bit-identical with vs without reset ? only the absolute timeline is
severed. The
623      # quality-floor / characterization snapshot is therefore unchanged (asserted in the
tests).
624      if time_origin_reset:
625          t0_frame = min(int(p.frame) for p in points)
626          t0_sec = t0_frame / float(fps)
627      else:
628          t0_frame, t0_sec = 0, 0.0
629      rows = _reset_trajectory_rows(points, t0_frame)
630      reset_gts = _reset_labels(gts, t0_sec)
631
632      # --- (e) write the de-attributed fixture (REUSE M1 writers + replay_fixture)
633      fdir = fixtures_dir / out_slug
634      fdir.mkdir(parents=True, exist_ok=True)
635      write_trajectory_csv(rows, fdir / "trajectory.csv")
636      write_labels_csv(reset_gts, fdir / "labels.csv")
637
638      iou = 0.5
639      prov = {

```

#### 4. user (2026-06-16T06:35:58.578Z)

```

780      out: List[str] = []
781
782      def exact(path: str, a: Any, b: Any) -> None:
783          if a != b:
784              out.append(f"{path}: {a!r} != {b!r} (exact)")
785
786      def approx(path: str, a: Any, b: Any) -> None:
787          try:
788              if abs(float(a) - float(b)) > _FLOAT_TOL:
789                  out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
790          except (TypeError, ValueError):
791              out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")
792
793      exact("slug", recomputed.get("slug"), committed.get("slug"))
794      exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
795      exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
796      approx("iou", recomputed.get("iou"), committed.get("iou"))
797
798      rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
799      if len(rc) != len(cc):
800          out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
801      else:
802          for i, (r, c) in enumerate(zip(rc, cc)):
803              approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
804              approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
805              rs, cs = r.get("shuttle", {}), c.get("shuttle", {})

```

```

806         if set(rs) != set(cs):
807             out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
808         else:
809             for k in rs:
810                 if k == "net_crossings":
811                     exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
812                 else:
813                     approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
814
815         rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
816         if len(rseg) != len(cseg):
817             out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
818         else:
819             for i, (r, c) in enumerate(zip(rseg, cseg)):
820                 approx(f"control_segments[{i}][0]", r[0], c[0])
821                 approx(f"control_segments[{i}][1]", r[1], c[1])
822
823         rsc, csc = recomputed.get("score", {}), committed.get("score", {})
824         int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
825         float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
"mean_start_error", "mean_end_error")
826         for k in int_fields:
827             exact(f"score.{k}", rsc.get(k), csc.get(k))
828         for k in float_fields:
829             approx(f"score.{k}", rsc.get(k), csc.get(k))
830
831         return out
832
833
834
835 # =====
836 # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
837 # =====
838
839
840 def _write_json(path: Path, obj: Any) -> None:
841     text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
842     path.write_text(text + "\n", encoding="utf-8", newline="\n")
843
844
845 def _read_json(path: Path) -> Any:
846     with open(path, encoding="utf-8") as f:
847         return json.load(f)
848
849
850 def _resolve_dir(slug_or_dir: Any) -> Path:
851     if isinstance(slug_or_dir, Path):
852         return slug_or_dir
853     p = Path(str(slug_or_dir))
854     if p.is_dir():
855         return p
856     return FIXTURES_DIR / str(slug_or_dir)
857
858
859 def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
860     if prov and "fps" in prov and "frame_width" in prov:
861         return float(prov["fps"]), float(prov["frame_width"])
862     spec = _spec_by_slug().get(slug)
863     if spec is not None:
864         return spec.fps, spec.frame_width
865     raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
866

```

```

867
868 def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
869     prov_path = fdir / "provenance.json"
870     prov = _read_json(prov_path) if prov_path.is_file() else None
871     return _fps_fw(prov, slug)
872
873
874 # =====
875 # CLI
876 # =====
877
878
879 def _cmd_check(fixtures_dir: Path) -> int:
880     manifest = load_manifest(fixtures_dir)
881     if not manifest:
882         print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
883               file=sys.stderr)
884         return 1
885     failures = 0
886     for entry in manifest:
887         slug = entry["slug"]
888         committed = _read_json(fixtures_dir / slug / "expected.json")
889         recomputed = replay_fixture(fixtures_dir / slug)
890         mismatches = compare_expected(recomputed, committed)
891         if mismatches:
892             failures += 1
893             print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
894                   f"(characterization = behaviour-locking, NOT correctness ? $6):")
895             for m in mismatches:
896                 print(f"    - {m}")
897         else:
898             sc = recomputed["score"]
899             print(f"[ok] {slug}: {len(recomputed['candidates'])} candidates, "
900                   f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")

```

## 5. assistant (2026-06-16T06:36:02.832Z)

The verifications confirm the key claims. Let me verify a few more specifics that the verifications didn't fully cover ? the CI matrix, the doc claims, and the manifest/provenance test assertions ? to triage the remaining findings accurately.

## 6. user (2026-06-16T06:36:03.887Z)

```

100 # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
101 # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
102 # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
103 # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
104 # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
105 golden-crossos:
106     name: Golden fixtures cross-OS (${{ matrix.os }})
107     strategy:
108         fail-fast: false
109         matrix:
110             os: [ubuntu-latest, windows-latest, macos-latest]
111     runs-on: ${{ matrix.os }}
112     steps:
113         - uses: actions/checkout@v6
114         - uses: actions/setup-python@v6
115           with:
116             python-version: "3.13"
117         - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
118           run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
119 genai numpy
120         - name: Golden-fixtures characterization (cross-OS determinism guard)
121           run: |

```

```
121         python -m backend.eval.golden_fixtures --check
122         python -m pytest tests/test_golden_fixtures.py -q
123
```

## 7. user (2026-06-16T06:36:04.832Z)

```
60
61     expected = gf.load_fixture(slug)["expected"]
62     for i, cand in enumerate(expected["candidates"]):
63         # The only feature block is the shuttle block, and it holds EXACTLY the 5
shuttle cols.
64         assert set(cand.keys()) == {"start", "end", "shuttle"}, f"{slug}: candidate[{i}]
has extra keys"
65         assert set(cand["shuttle"].keys()) == set(gf.SHUTTLE_FEATURES), (
66             f"{slug}: candidate[{i}].shuttle is not exactly the 5 shuttle features")
67
68
69     @pytest.mark.parametrize("slug", SLUGS)
70     def test_synthetic_provenance_tags(slug):
71         """Tier-0 hard tags: kind=synthetic, license=synthetic, minors_free=true (constraint
5)."""
72         prov = gf.load_fixture(slug)["provenance"]
73         assert prov["kind"] == "synthetic"
74         assert prov["license"] == "synthetic"
75         assert prov["minors_free"] is True
76         # Windowing pinned in the fixture (not the named SERVED constant) ? constraint 3.
77         assert set(prov["windowing"]) == {"vt", "mg", "mwd", "max_win"}
78
79
80     def test_control_is_not_learned():
81         """The replay must use the pure-stdlib CONTROL variant (no numpy/LogisticRegression
? D5)."""
82         from backend.eval.experiment import CONTROL
83         assert CONTROL.is_learned() is False
84
85
86     def test_perturbing_trajectory_changes_segments(tmp_path):
87         """Mutating a copy of a trajectory and replaying yields DIFFERENT segments ? proving
the
88         windows are RE-DERIVED from the CSV each run, not merely echoed from
expected.json."""
89         slug = SLUGS[0]
```

## 8. user (2026-06-16T06:36:06.025Z)

```
940         ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
941             help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
942         args = ap.parse_args(list(argv) if argv is not None else None)
943
944         fixtures_dir: Path = args.fixtures_dir
945
946         if args.freeze_synthetic:
947             slugs = freeze_synthetic(fixtures_dir)
948             print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'',
'.join(slugs)}")
949             print(f"  -> {fixtures_dir}")
950             return 0
951
952         if args.refresh_snapshot:
953             if not args.slug and not args.all:
954                 ap.error("--refresh-snapshot requires --slug S or --all")
955             if not args.reason:
956                 print("[golden-fixtures] WARNING: no --reason given. --refresh-snapshot can
"
```

```

957             "rubber-stamp a real regression ? review the visible diff ($6).",
file=sys.stderr)
958         targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
959         print(f"[golden-fixtures] refreshed snapshot for: {' ', '.join(targets)}"
960               + (f" (reason: {args.reason})" if args.reason else ""))
961         return 0
962
963     if args.freeze_real:
964         missing = [n for n, v in (("--trajectory", args.trajectory),
965                                  ("--labels", args.labels),
966                                  ("--fps", args.fps),
967                                  ("--frame-width", args.frame_width)) if v is None]
968         if missing:
969             ap.error("--freeze-real requires " + " ".join(missing))
970         try:
971             expected = freeze_real_fixture(
972                 args.label, args.trajectory, args.labels,
973                 license=args.license_ if args.license_ is not None else "",
974                 minors_free=args.minors_free, owned=args.owned,
975                 fps=args.fps, frame_width=args.frame_width,
976                 fixtures_dir=fixtures_dir, slug=args.slug,
977                 time_origin_reset=args.time_origin_reset, source_note=args.source_note)
978         except RefusalError as e:
979             # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-
zero.
980             print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
file=sys.stderr)
981             return 2
982             print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'"
"
983                 f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
984             print(" $6: behaviour-locking, NOT correctness; non-attributable only relative
to the "
985                   "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
(P2).")
986             return 0
987
988     # --check
989     return _cmd_check(fixtures_dir)

```

## 9. assistant (2026-06-16T06:36:08.808Z)

Now I have confirmed all the key facts. Let me verify two remaining specifics: the doc \$4c/\$4a claims and the schema\_version guard in load\_fixture, to finalize triage of those findings.

## 10. user (2026-06-16T06:36:11.164Z)

```

755     def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
756         """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses
a fixture
757         whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible)."""
758         fdir = fixtures_dir / slug
759         prov = _read_json(fdir / "provenance.json")
760         expected = _read_json(fdir / "expected.json")
761         sv = int(expected.get("schema_version", prov.get("schema_version", 0)))
762         if sv > CURRENT_SCHEMA:
763             raise ValueError(
764                 f"{slug}: schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA} ? upgrade
the harness")
765         return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
766
767
768     # =====
769     # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats

```

## 11. user (2026-06-16T06:36:12.816Z)

```
80: `parse_trajectory_csv` ? windowing.build_windows(preset) ? distill_local.build_candidates ?  
rally_gate net_crossings ? experiment.predict(CONTROL) ? merge_overlaps ? metrics.score`.  
81: Reuses verbatim: `metrics.score/interval_iou`, `segmentation_metrics`,  
`experiment.predict/compare/Variant.spec_hash`, `windowing.WindowingPreset` (default SERVED),  
`regression.gt_hash/save_baseline + incommensurability guards`, `eval_stats` (seed=0),  
`eval_baselines/` committed-baseline precedent.  
96: - Raw-trajectory exclusion ? never commit the per-frame `(Frame,Visibility,X,Y)` CSV.  
134: - --refresh-snapshot can rubber-stamp a real regression; mitigation is review of the  
visible diff + a separate, deliberate floor-baseline update. Require a recorded reason.
```

## 12. assistant (2026-06-16T06:36:26.975Z)

All key facts are now confirmed against the code. I have enough to triage. Let me synthesize.

Summary of confirmation status:

- **freeze\_real\_fixture** line 635\*\* writes full per-frame `trajectory.csv` ? confirmed (raw CSV committed, contradicting doc §4c:96).
- **--scan\_forbidden** lines 501-535\*\* ? no free-text scan; `source\_note`/`label` written verbatim ? confirmed.
- **CI ci.yml:119-122** ? golden-crossos runs only **--check** + `test\_golden\_fixtures.py`; `test\_golden\_real\_fixtures.py` not in matrix ? confirmed.
- **Time-origin reset** non-invariance for sub-second t0 ? confirmed by verification; but no committed cleared\_real fixtures and **--check** is self-consistent.
- **compare\_expected** lines 823-830\*\* ? fixed-field lists, extra score/top-level key ignored ? confirmed.
- **--scan\_forbidden** line 530\*\* ? substring skip over-skips (`label`, `traj` etc.) ? confirmed in code.
- **refresh\_snapshot** main() line 955-957\*\* ? warns, doesn't require **--reason**; docstring says "requires" ? confirmed.
- **test\_synthetic\_provenance\_tags** lines 70-77\*\* ? hard-asserts `kind=='synthetic'` over ALL manifest slugs; will red on cleared\_real ? confirmed.
- **load\_fixture** line 762\*\* ? only guards forward-incompat; prov-only forward version (expected has field) reads expected's value as priority ? minor.
- **numpy import** ? transitive import-time dep, "pure-stdlib/no numpy" over-claims ? confirmed.
- **W1/max\_win** ? pinned but unthreaded through replay ? confirmed by robustness audit.
- **Doc §4a:80-81** ? names `windowing.build\_windows`/`WindowingPreset`/"default SERVED" not on replay path ? confirmed.

Now triaging. The critical question per the prompt: only CONFIRMED real defects in mustFix. The verifications mark only ONE finding as `isMustFix=true` (the free-text source\_note/label leak). The others are confirmed-but-deferrable (no real fixtures committed yet; nothing currently broken) or refuted/partial.

## 13. user (2026-06-16T06:37:36.121Z)

Structured output provided successfully

## 14. assistant (2026-06-16T06:37:48.771Z)

I've completed the triage. The structured output has been recorded.

**Verdict:** The shipped golden-fixtures feature **passes** due-diligence for the synthetic Tier-0 corpus actually committed today\*\* ? nothing currently relied upon is broken (green suite, self-consistent cross-OS **--check**, sound refusal gate and surrogate). But it is **not** clean for the de-attribution / real-data (P3) path\*\* it exists to enable.

**One mustFix (CONFIRMED, blocks P3/counsel sign-off):** **--scan\_forbidden** does not scan free-text `source\_note`/`label` ? a player/match/event name committed verbatim into provenance.json passes silently (empirically reproduced). This is the exact leak the refusal gate is meant to prevent.

**Key triage calls aligned with the verifications:**

- The verifications marked only the free-text leak `isMustFix=true` ? I matched that.
- The "raw per-frame CSV retains de Montjoye fingerprint" privacy alarm was **refuted** as a live leak (all committed fixtures are synthetic, machine-generated coords; no real fixture exists) ? but its real residue, the doc-vs-code contradiction on raw-CSV exclusion, is in niceToHave.
- The time-origin-reset non-invariance is genuinely confirmed but does **not** flake the suite or `--check` (self-consistent on already-reset bytes), so it's a deferrable correctness/doc fix, not a blocker.
- The loud-fail guarantees (FEATURE\_NAMES import assert, signature TypeErrors, exact-int net\_crossings, reordering/count drift) and the M2 gate / os.urandom surrogate are all real and went to nonIssues.

The cluster of confirmed-but-deferrable items (reset arithmetic, compare\_expected extra-key blind spot, scan substring over-skip, W1/max\_win pinned-but-ignored, the `test\_synthetic\_provenance\_tags` forward-trap, missing negative tests, and the numpy/SERVED/refresh-reason doc over-claims) should all be cleared before the real-data path is depended on, since the owner's 2026-06-16 risk acceptance was made against a doc that overstates the de-attribution actually implemented.